

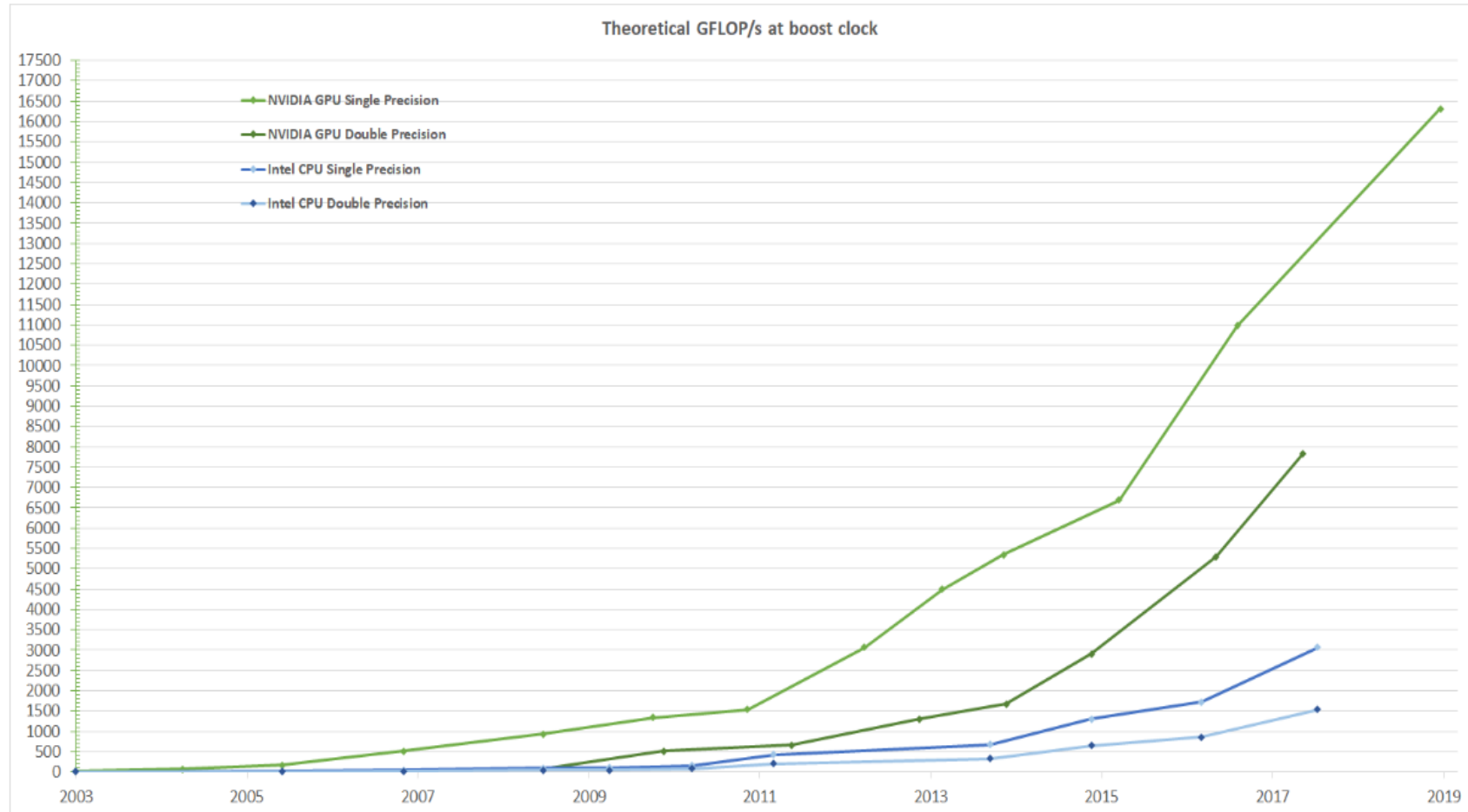
Overview of Cuda Programming

Jyothi Vedurada

(Sources: CS6023 GPU Programming course slides of Rupesh Nasre, IITM;
Cuda Programming Guide)

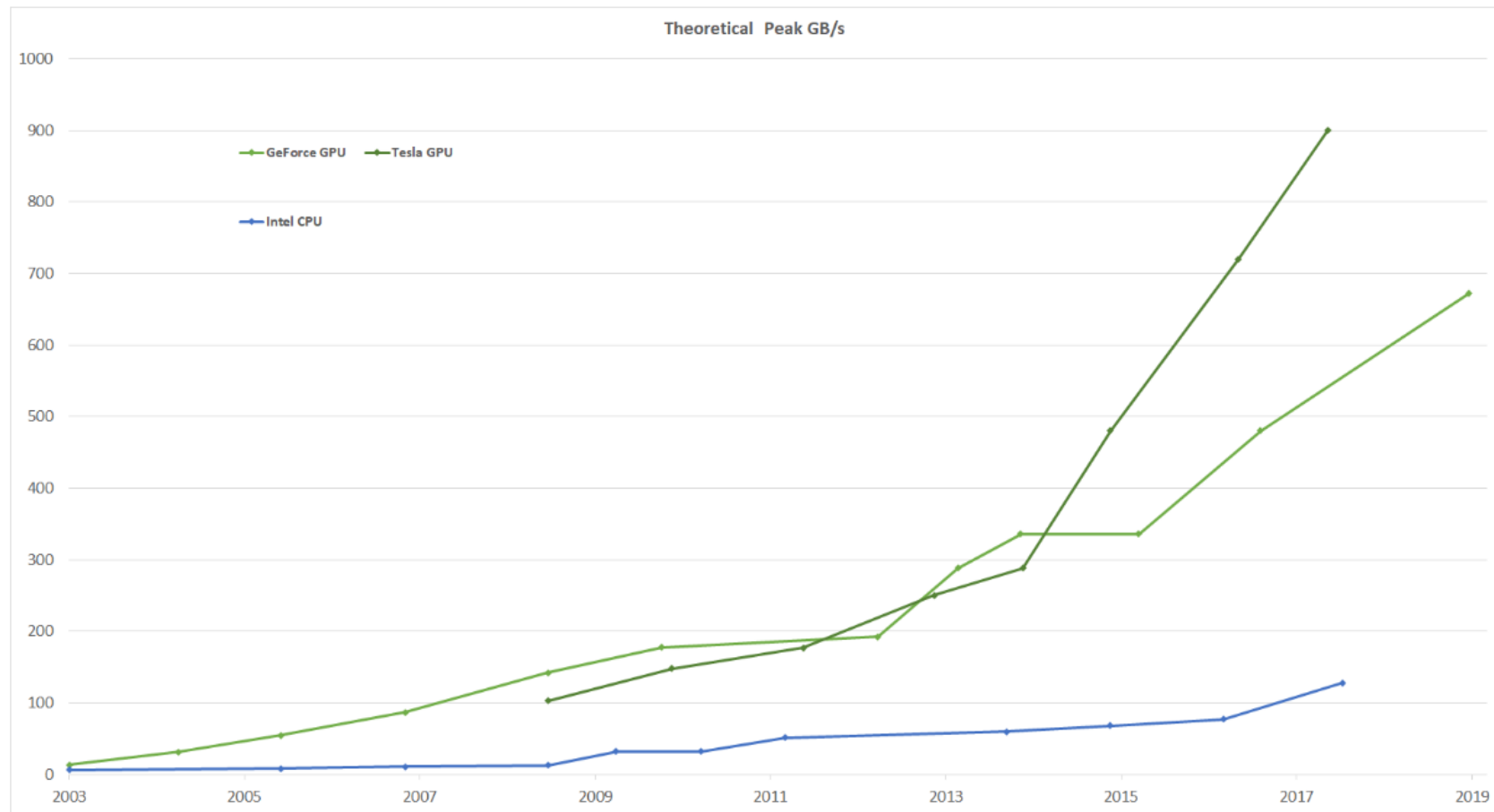
Why GPUs?

GFLOPS



Why GPUs?

Bandwidth



Difference between CPU and GPU



- Low number of cores(2-8)
- Complex flow control
- Large data and instruction cache
- Optimized for serial computing

- High number of cores (2048)
- Simple distributed control
- Minimal cache
- Optimized for data-parallel processing
- More transistor devoted to data processing

GPU Hello World

```
#include <stdio.h>
#include <cuda.h>

__global__ void dkernel() {
    printf("Hello World.\n");
}

int main() {
    dkernel<<1,1>>();
    cudaDeviceSynchronize();
    return 0;
}
```

CPU function and GPU Kernel
run asynchronously

Another CUDA Program

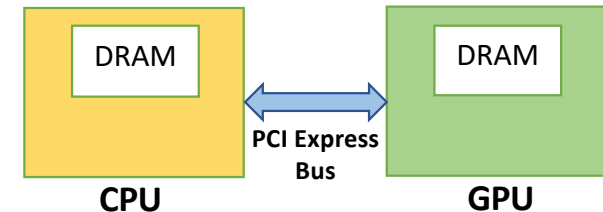
```
#include <stdio.h>
#define N 100

int main() {
    int a[N], i;
    for (i = 0; i < N; ++i)
        a[i] = i * i;
    return 0;
}
```

```
#include <stdio.h>
#define N 100
```

```
__global__ void fun(int *a) {
    a[threadIdx.x] = threadIdx.x * threadIdx.x; }
```

```
int main() {
    int a[N], *da; int i;
    cudaMalloc(&da, N * sizeof(int));
    fun<<>>(da);
    cudaMemcpy(a, da, N * sizeof(int), cudaMemcpyDeviceToHost);
    for (i = 0; i < N; ++i)
        printf("%d\n", a[i]);
    return 0;
}
```

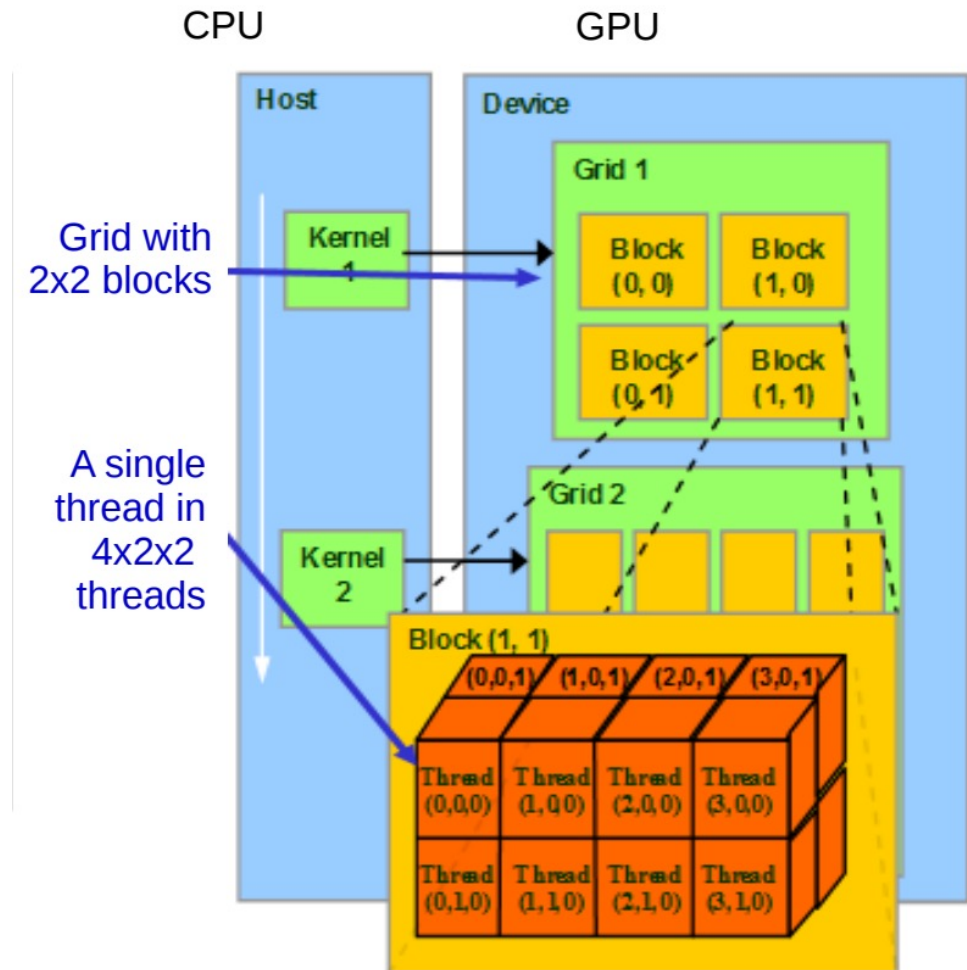


Typical CUDA Program Flow

1. Load data into CPU memory.
 - fread
2. Copy data from CPU to GPU memory.
 - cudaMemcpy(..., cudaMemcpyHostToDevice)
3. Call GPU kernel.
 - mykernel<>>(...)
4. Copy results from GPU to CPU memory.
 - cudaMemcpy(..., cudaMemcpyDeviceToHost)
5. Use results on CPU.

Thread Organization: Grids, Blocks, Threads

- Each thread uses IDs to decide what data to work on.
 - Block ID: 1D, 2D, or 3D
 - Thread ID: 1D, 2D, or 3D
- Simplifies memory addressing when processing multi-dimensional data
- Typical configuration:
 - 1-5 blocks per SM
 - 128-1024 threads per block
 - Total 2K-100K threads
- You can launch a kernel with millions of threads.



Thread Organization: Grids, Blocks, Threads

```
#include <stdio.h>
#include <cuda.h>

__global__ void dkernel() {
    if (threadIdx.x == 0 && blockIdx.x == 0 &&
        threadIdx.y == 0 && blockIdx.y == 0 &&
        threadIdx.z == 0 && blockIdx.z == 0) {
        printf("%d %d %d %d %d %d.\n", gridDim.x,
            gridDim.y, gridDim.z, blockDim.x,
            blockDim.y, blockDim.z);
    }
}

int main() {
    dim3 grid(2, 3, 4);
    dim3 block(5, 6, 7);
    dkernel<<>>();
    cudaThreadSynchronize();
    return 0;
}
```

